

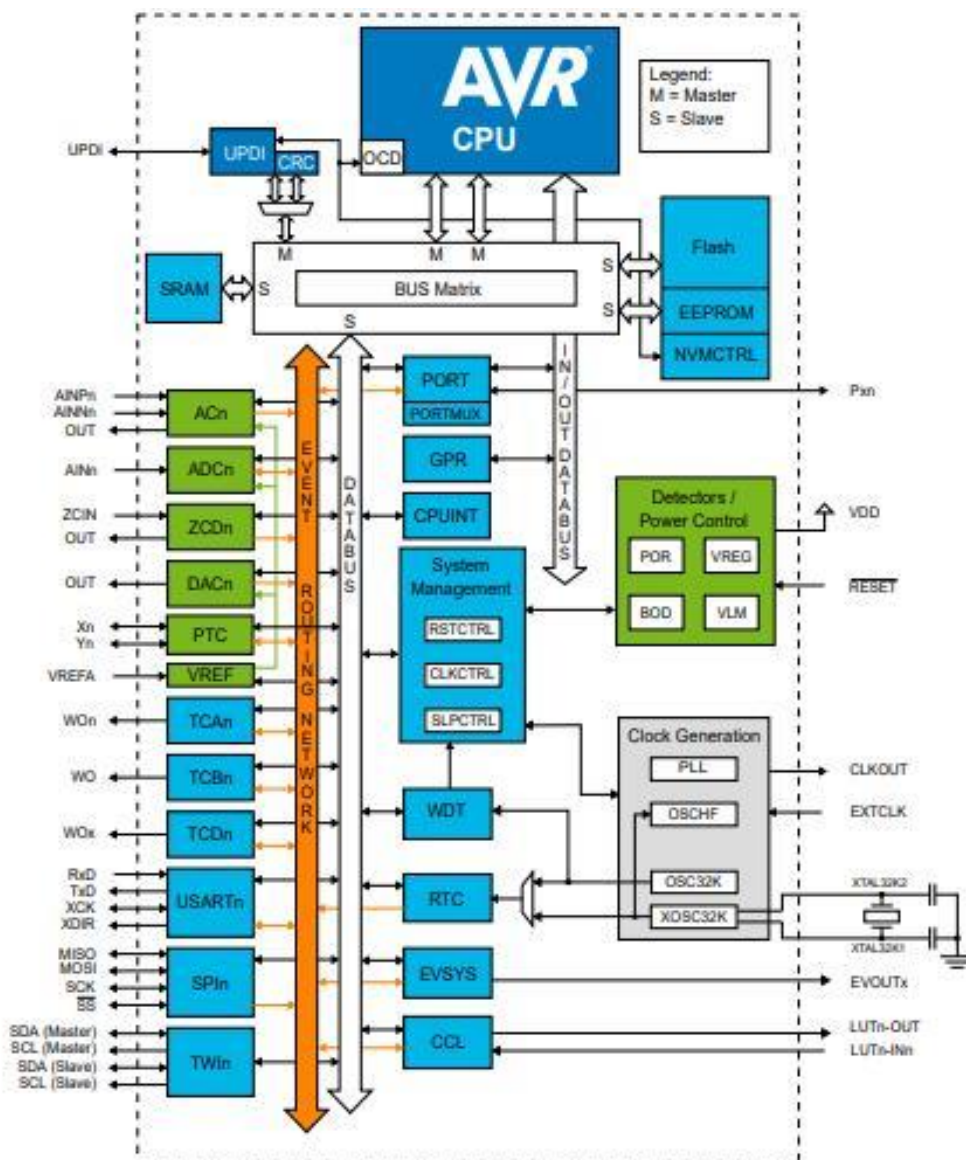
Unit-III: Introduction to AVR RISC Microcontroller Architecture

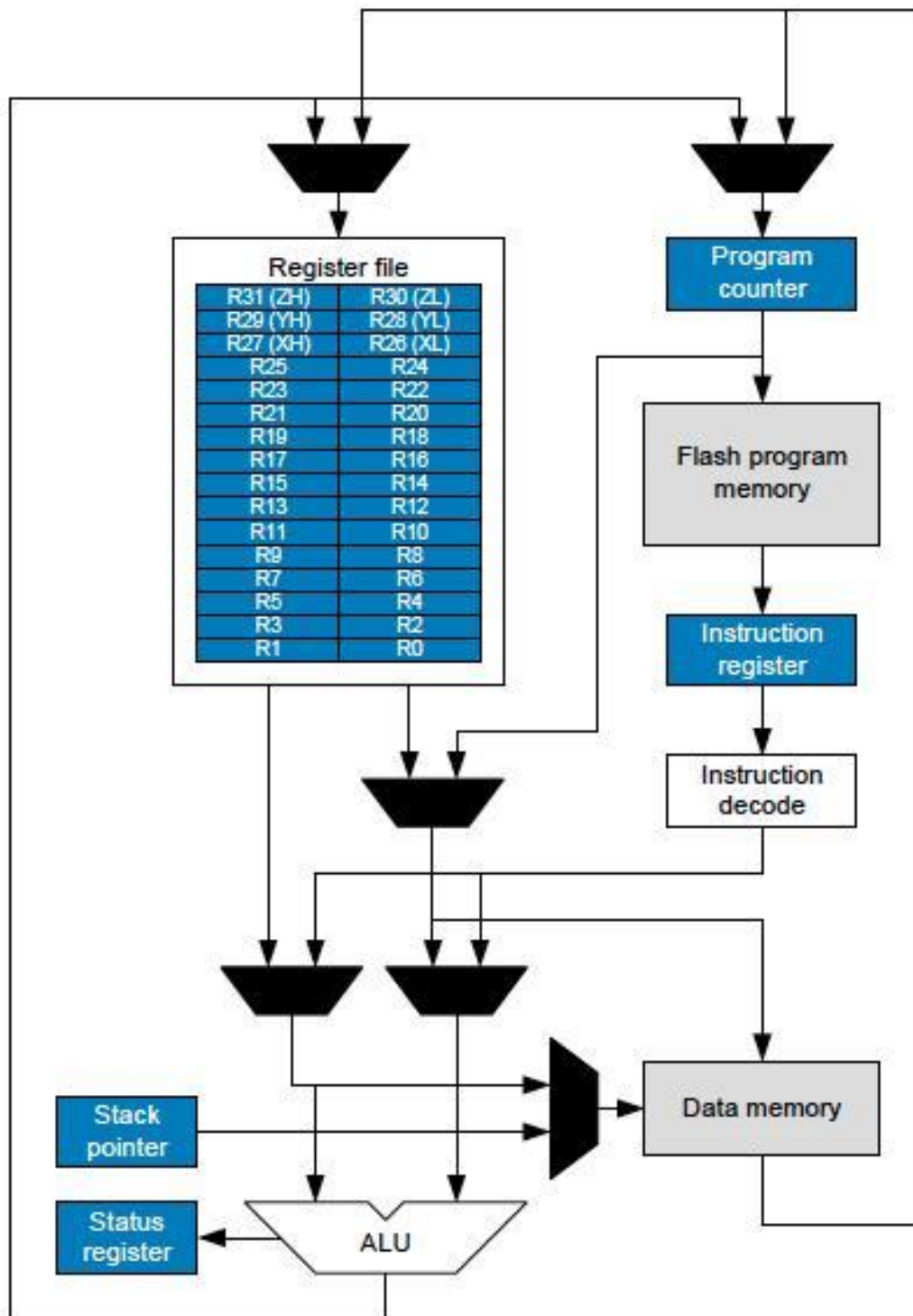
❖ What is AVR Microcontroller?

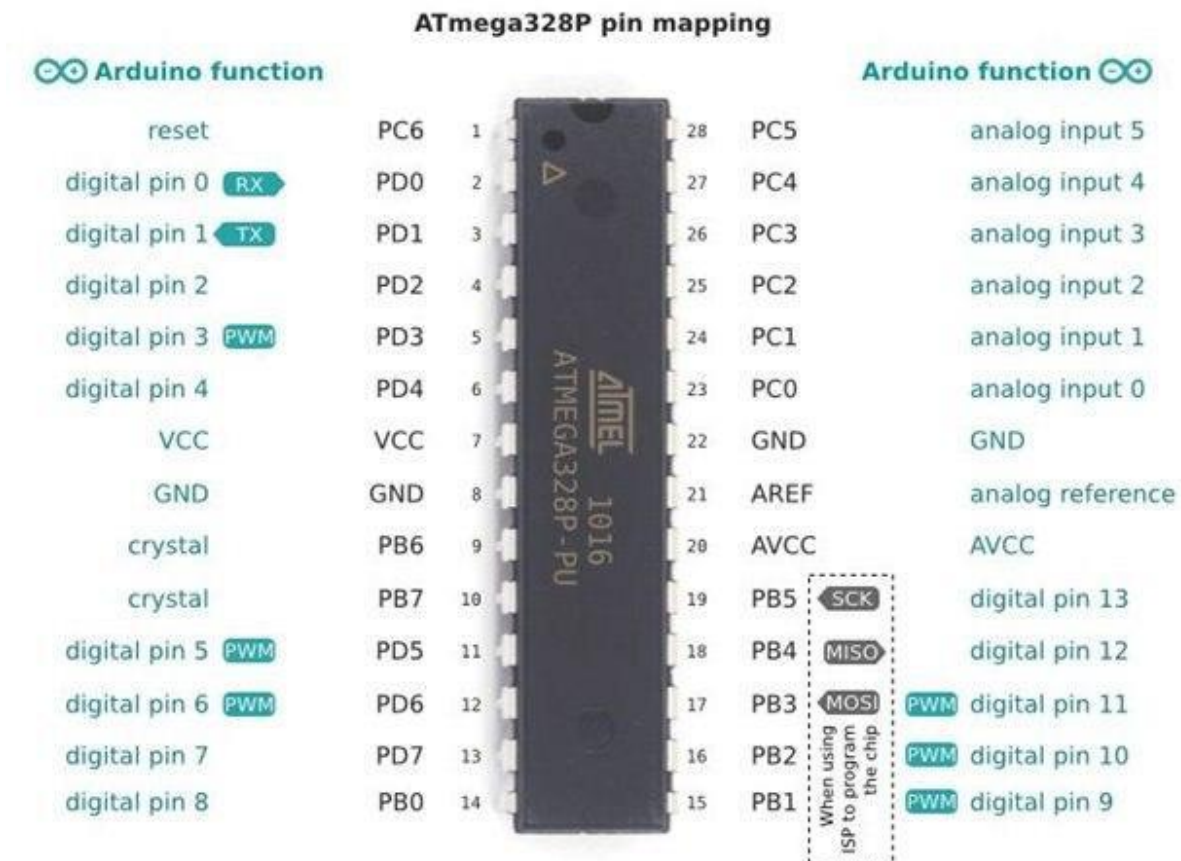
The **AVR microcontroller** is a family of 8-bit microcontrollers developed by Atmel (now part of Microchip Technology). It is widely used in embedded systems due to its **high performance, low power consumption, and simple architecture**.

AVR stands for:

- **A**: Alf (one of the designers)
- **V**: Vegard (co-designer)
- **R**: RISC (Reduced Instruction Set Computer)







2. RISC Architecture Concept

AVR is based on the **Reduced Instruction Set Computer** principle, which means:

- Uses a **small, simple set of instructions**
- Most instructions execute in **one clock cycle**
- Improves **speed and efficiency**

👉 This makes AVR faster compared to traditional CISC microcontrollers.

3. Key Features of AVR Architecture

a) Harvard Architecture

AVR follows **Harvard architecture**, meaning:

- Separate memory for **program (Flash)** and **data (SRAM)**
- Enables **simultaneous access** → faster execution

b) 8-bit CPU

- Processes **8 bits at a time**

- Suitable for **embedded applications** like sensors, automation, robotics

c) 32 General Purpose Registers

- AVR has **32 working registers (R0–R31)**
- Directly connected to ALU
- Reduces memory access time → improves speed

d) Single-Cycle Execution

- Most instructions execute in **one clock cycle**
- High throughput performance

e) Pipeline Architecture

- Uses **instruction pipelining**
 - Fetches next instruction while executing current one
-

4. Memory Organization

AVR microcontroller typically includes:

- **Flash Memory**
 - Stores program/code
 - Non-volatile
 - **SRAM**
 - Temporary data storage
 - Volatile
 - **EEPROM**
 - Stores permanent data (even after power off)
-

5. CPU Components

The AVR CPU mainly consists of:

- **ALU (Arithmetic Logic Unit)** – performs arithmetic and logical operations
 - **Register File (32 registers)** – fast data storage
 - **Program Counter (PC)** – holds address of next instruction
 - **Stack Pointer (SP)** – manages function calls and interrupts
-

6. I/O and Peripherals

AVR microcontrollers include built-in peripherals:

- Digital Input/Output ports
 - Timers/Counters
 - ADC (Analog to Digital Converter)
 - USART, SPI, I2C communication interfaces
 - Interrupt system
-

7. Advantages of AVR Architecture

- High speed (single-cycle execution)
- Low power consumption
- Easy to program (supports C, Assembly)
- Rich set of peripherals
- Cost-effective for embedded systems

8. Applications

AVR microcontrollers are widely used in:

- Embedded systems
- Robotics
- Industrial automation
- Consumer electronics
- IoT devices

❖ Register File

What is a Register File?

A **register file** is a small, fast storage area inside a CPU or microcontroller that contains a set of **registers** used for temporary data storage during instruction execution.

- It is part of the **CPU core**
 - Provides **high-speed access** compared to RAM
 - Used to store **operands, intermediate results, and control data**
-

2. Register File in AVR Microcontroller

In the AVR architecture, the register file is one of the most important features.

Key Characteristics:

- **32 general-purpose registers:** R0 to R31

- Each register is **8 bits wide**
- Directly connected to the **ALU (Arithmetic Logic Unit)**
- Enables **single-cycle instruction execution**

3. Structure of Register File

The AVR register file is organized as:

- **R0 – R15** → General-purpose registers
- **R16 – R31** → Used for immediate operations and advanced instructions

Additionally, some registers are paired for special operations:

Register Pair Function

R26–R27 X register (Pointer)

R28–R29 Y register (Pointer)

R30–R31 Z register (Pointer)

These are used for:

- Indirect addressing
- Accessing memory locations

4. Functional Diagram (Concept)

A register file typically includes:

- **Read ports** → To fetch data from registers
- **Write port** → To store data into registers
- **Decoder** → Selects which register to access
- **ALU connection** → Direct data path for fast execution

6. Advantages of Register File

- ⚡ **Very fast access** (single clock cycle)
- 🔄 **Supports parallel operations**
- 🧠 **Reduces memory access**, improving performance
- 🖱️ **Direct connection to ALU** enhances speed

7. Limitations

- Limited storage (only 32 registers in AVR)
- Cannot store large data like RAM

- Requires efficient usage by programmer/compiler
-

8. Importance in Embedded Systems

In embedded systems (like AVR-based systems), the register file:

- Improves **execution speed**
- Enables **efficient instruction processing**
- Plays a crucial role in **real-time applications**

❖ Memory Organization

Memory organization defines how memory is **structured, accessed, and managed** inside a microcontroller or computer system. In embedded systems—especially the AVR Microcontroller—memory is carefully designed for **speed, efficiency, and real-time operation**.

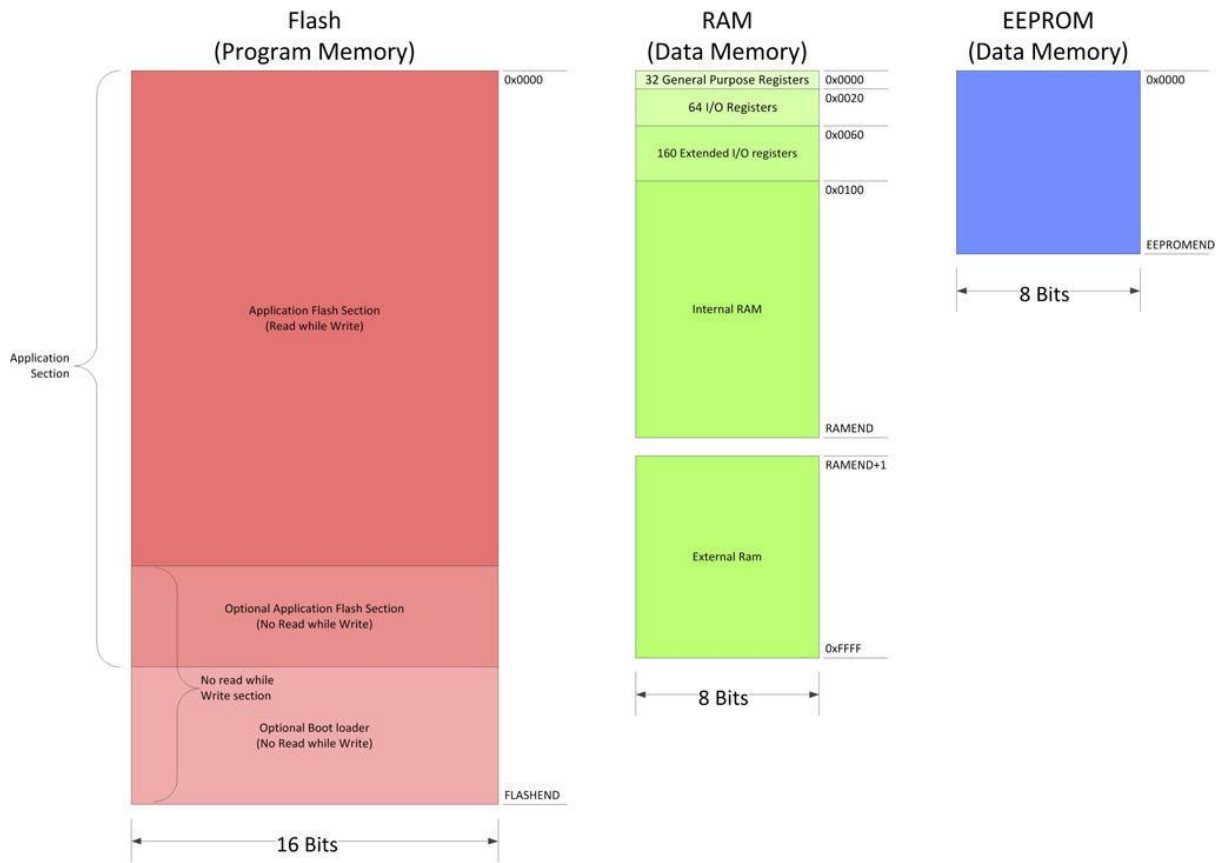
1. Types of Memory in AVR

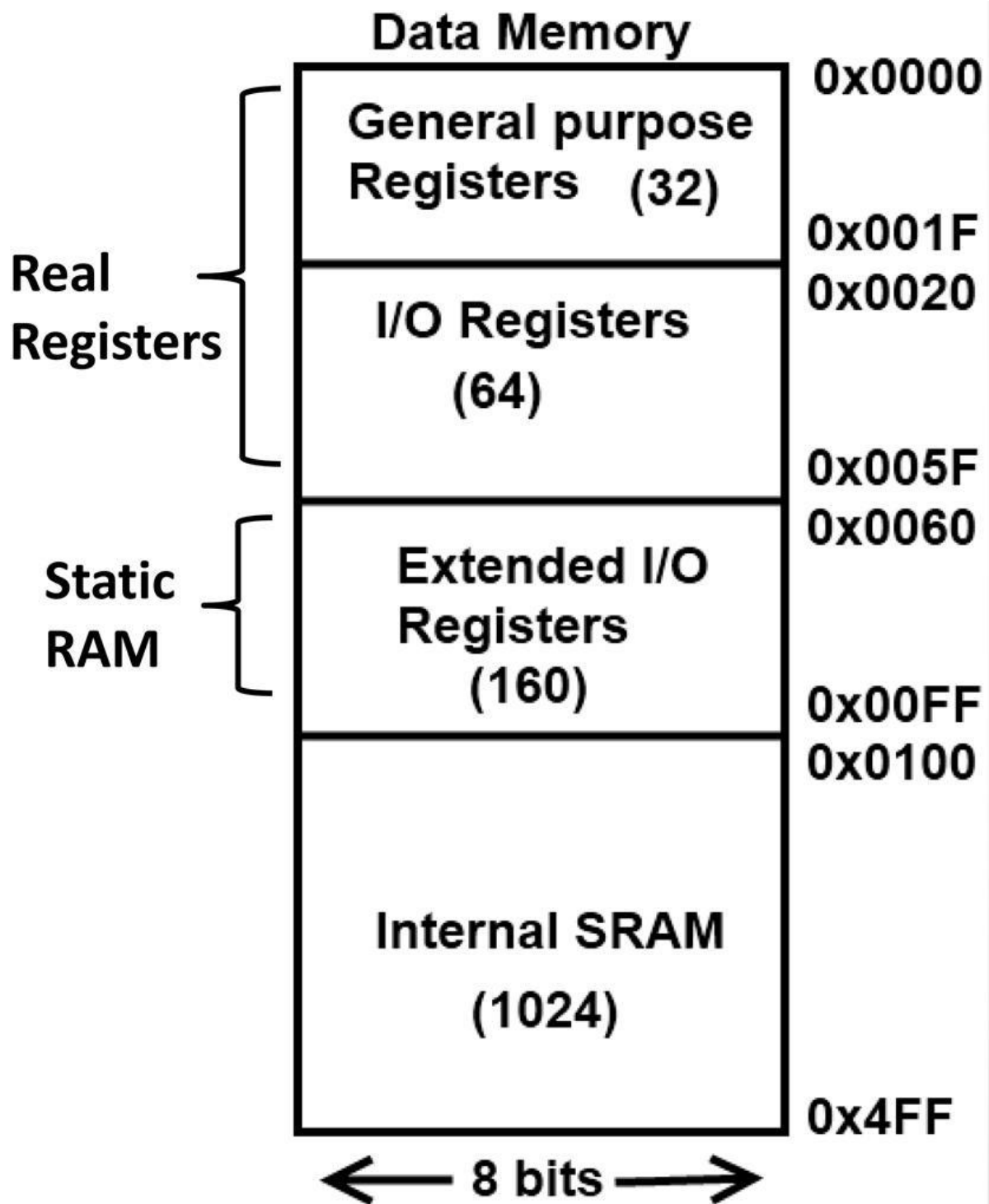
AVR follows a **Harvard Architecture**, meaning **separate memory spaces** for program and data.

◆ Main Memory Types:

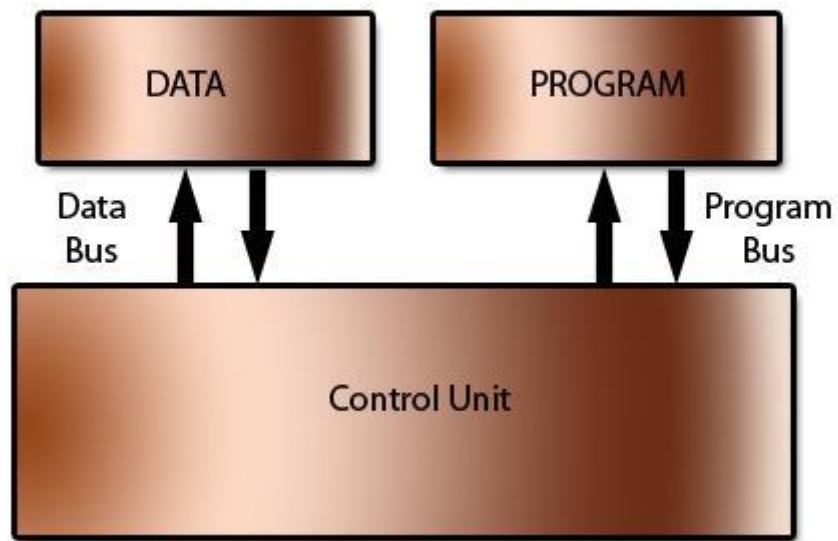
1. **Program Memory (Flash)**
 2. **Data Memory (SRAM)**
 3. **EEPROM (Non-volatile Data Memory)**
-

2. Program Memory (Flash Memory)





The Harvard architecture



Key Features:

- Stores **program instructions (code)**
- **Non-volatile** (retains data without power)
- Organized as **words (16-bit instructions)**
- Accessed using the **Program Counter (PC)**

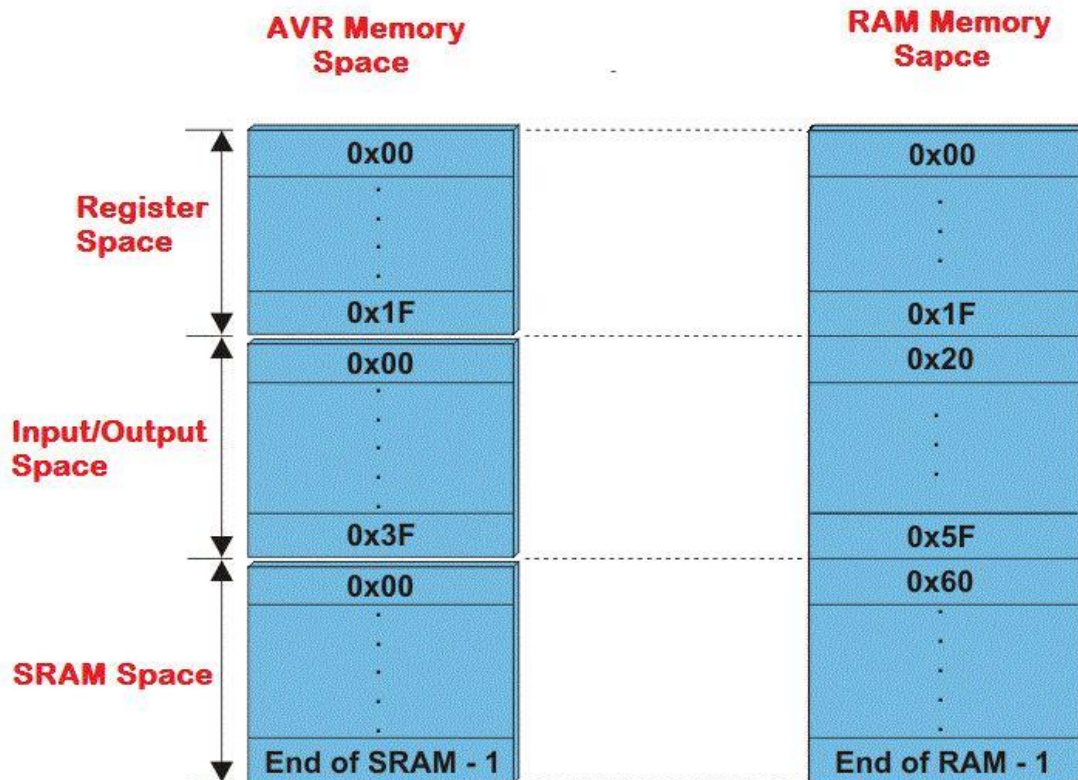
Working:

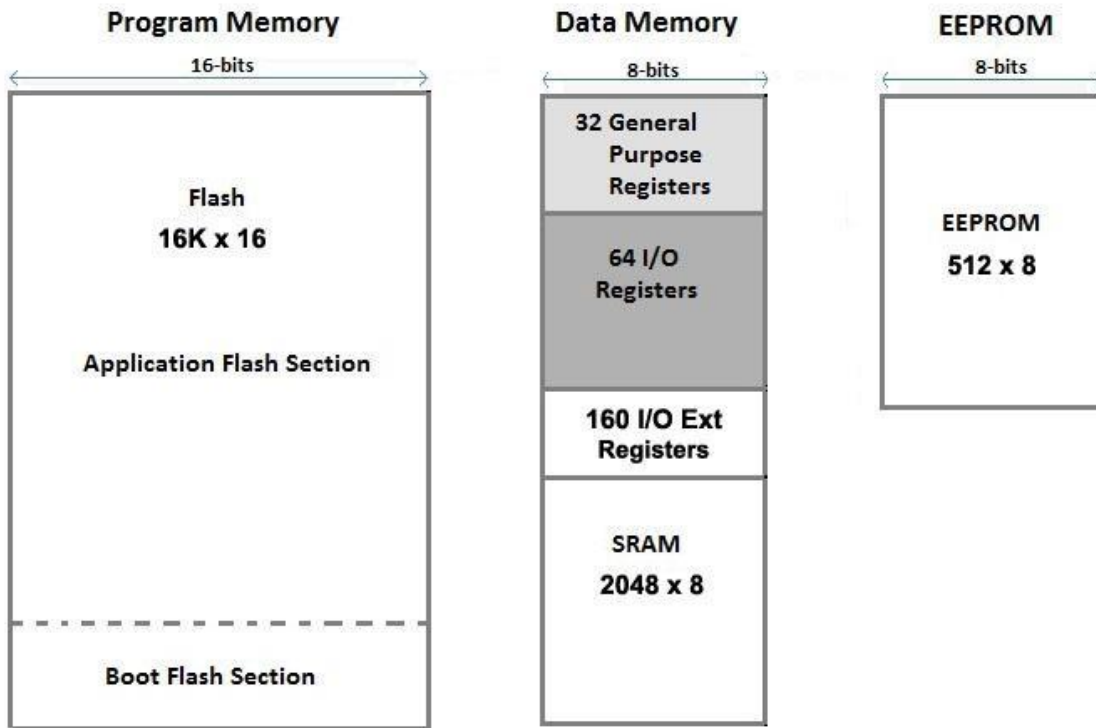
- CPU fetches instructions sequentially from Flash
- PC increments automatically after each instruction
- Supports **branching and looping**

3. Data Memory (SRAM)

IN/OUT		Load/Store
0x0000 – 0x001F	32 registers	0x0000 – 0x001F
	64 I/O registers	0x0020 – 0x005F
	160 Ext I/O registers	0x0060 – 0x00FF
	Internal SRAM (2048x8)	0x0100 0x08FF

AVR Memory Spaces





Structure of Data Memory:

Data memory in AVR is divided into **three parts**:

1. Register File (0x00 – 0x1F)

- 32 general-purpose registers (R0–R31)
- Fastest access

2. I/O Registers (0x20 – 0x5F)

- Used for controlling peripherals
- Examples: PORT, DDR, PIN registers

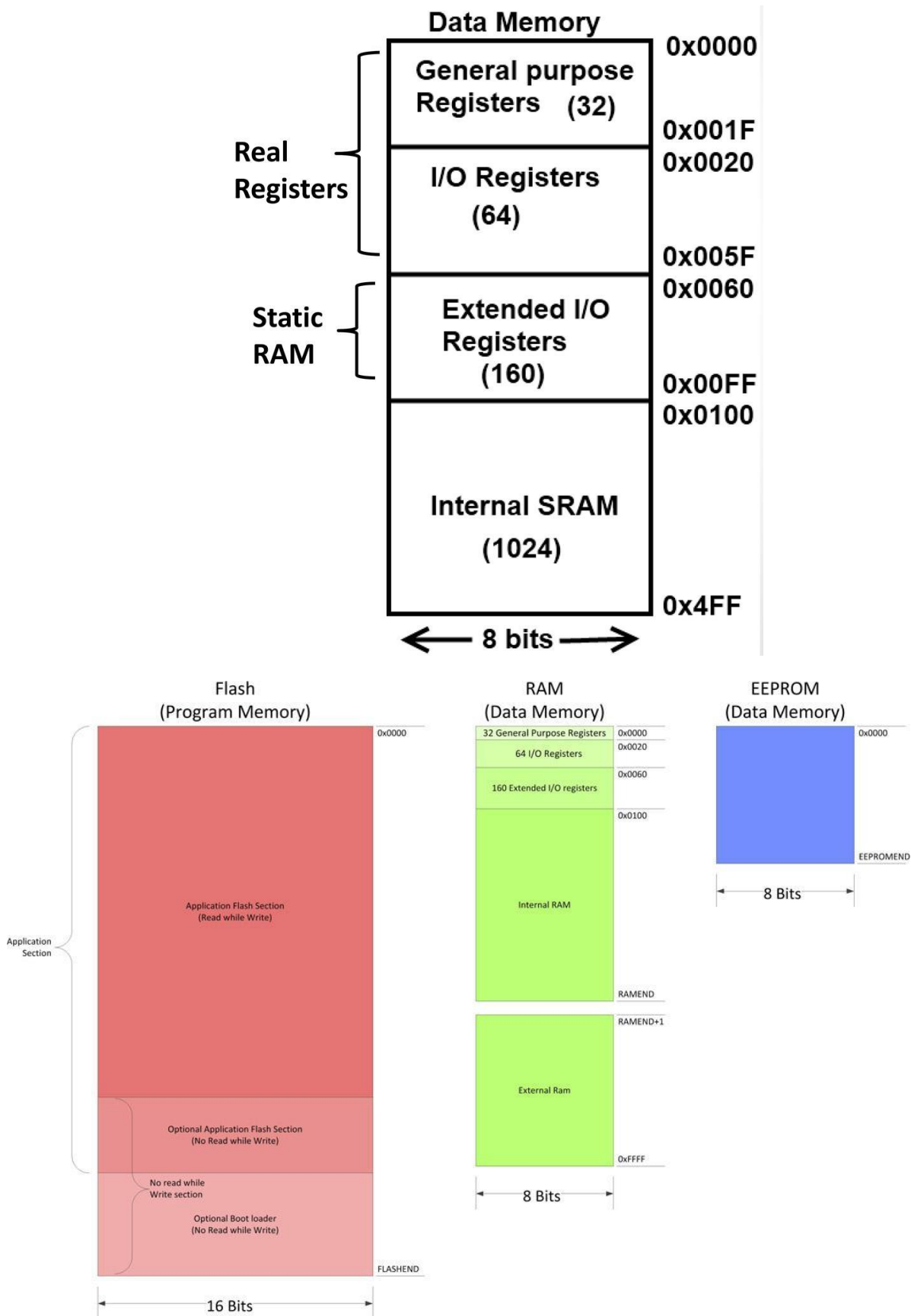
3. Internal SRAM (0x60 onwards)

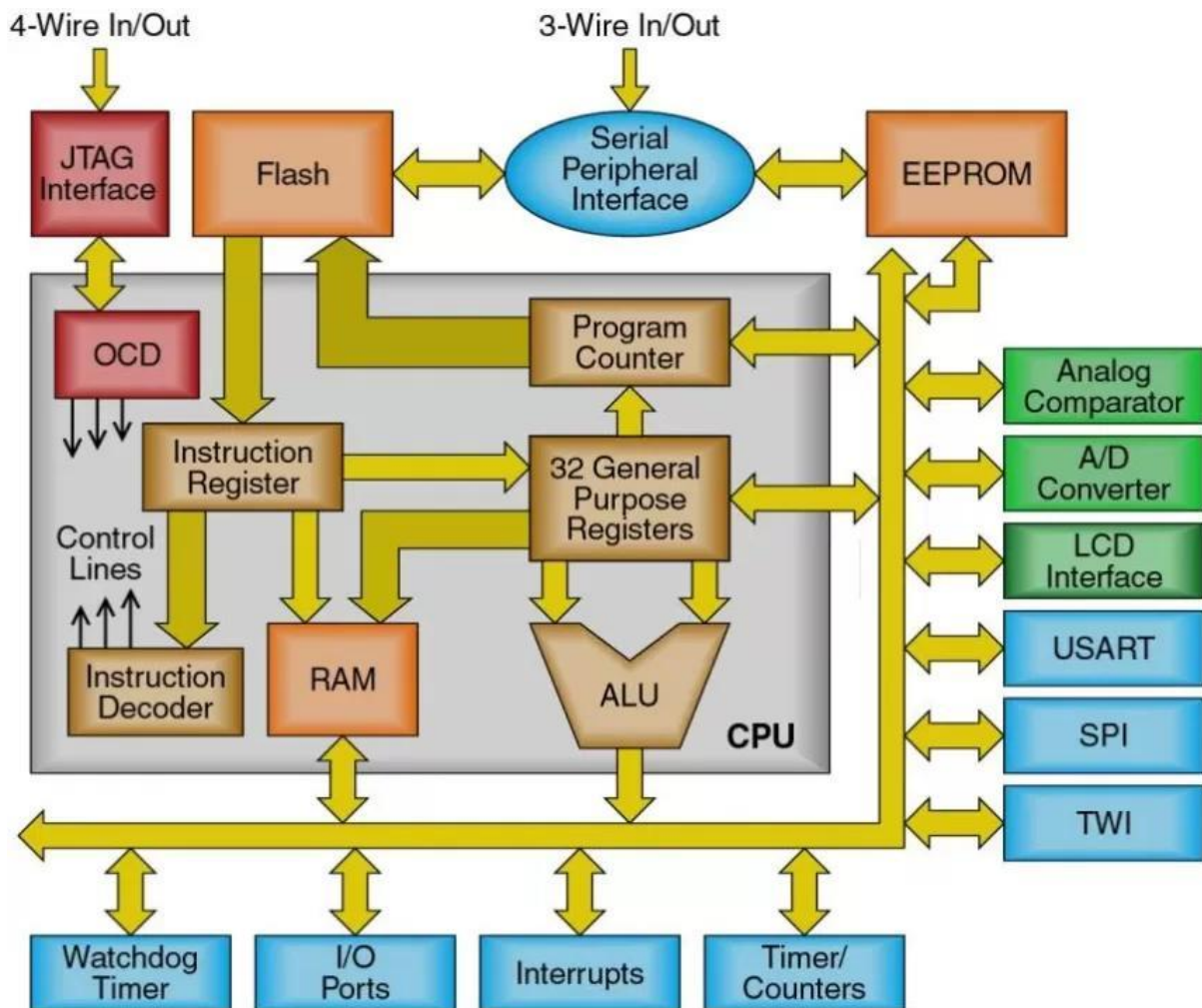
- Used for:
 - Variables
 - Stack
 - Temporary data storage

Stack in SRAM:

- Used for **function calls and interrupts**
- Managed using **Stack Pointer (SP)**
- Works in **LIFO (Last In First Out)** manner

4. EEPROM (Electrically Erasable Programmable ROM)





Key Features:

- Non-volatile memory for **permanent data storage**
- Used to store:
 - Configuration settings
 - Calibration data
- Slower than SRAM
- Supports **byte-wise read/write**

5. Memory Map of AVR

The memory map shows how memory is arranged:

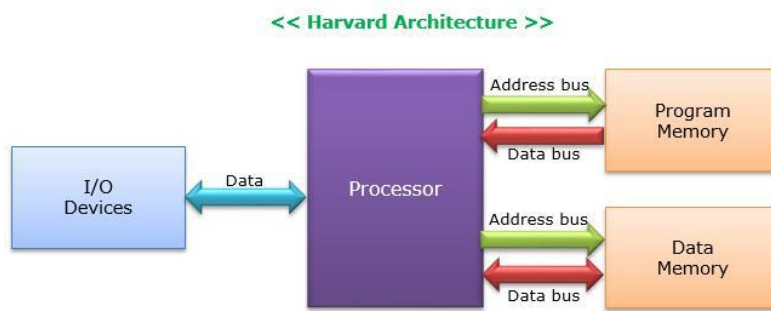
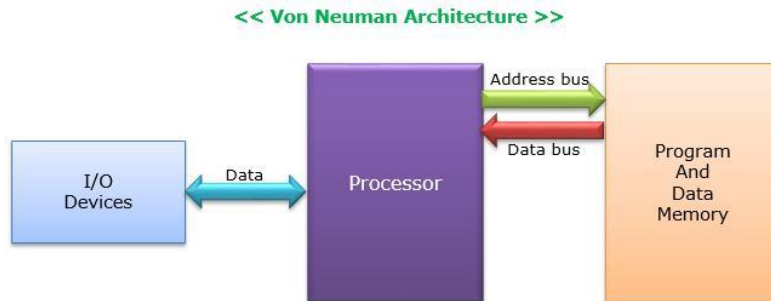
Memory Type Address Range Purpose

Register File	0x00 – 0x1F	CPU registers
I/O Registers	0x20 – 0x5F	Peripheral control
SRAM	0x60 – End	Data storage
Flash Memory	Separate space	Program code

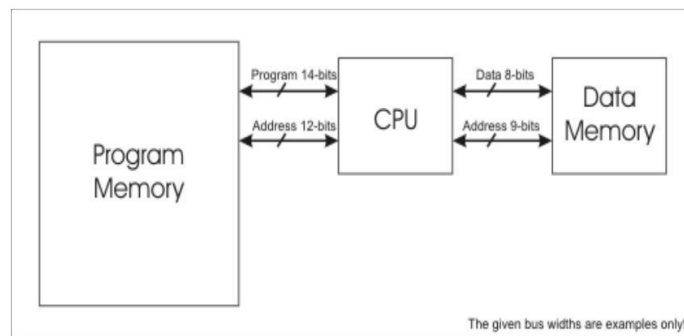
Memory Type Address Range Purpose

EEPROM Separate space Non-volatile data

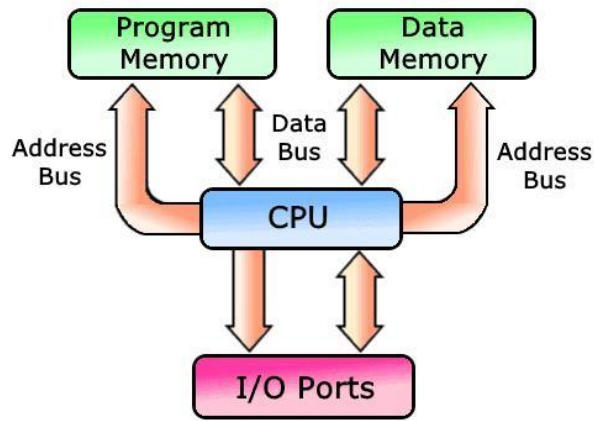
6. Harvard Architecture Concept



AVR Microcontroller



Harvard Architecture diagram



Harvard Architecture

Key Idea:

- Separate buses for:
 - Program memory
 - Data memory

Advantages:

- ⚡ Faster execution (parallel access)
- 🚀 Efficient instruction processing
- 🔄 Simultaneous read/write operations

7. Addressing Modes in Memory Access

AVR supports multiple ways to access memory:

- **Immediate Addressing** → Data is part of instruction
- **Direct Addressing** → Direct memory location
- **Indirect Addressing** → Using pointer registers (X, Y, Z)
- **Indexed Addressing** → Base + offset

8. Advantages of AVR Memory Organization

- Fast execution due to Harvard architecture
- Efficient use of memory resources
- Flexible data handling using multiple memory types
- Supports real-time embedded applications

9. Limitations

- Limited SRAM size
- EEPROM is slower

- Requires careful memory management

❖ I/O Ports

In the AVR Microcontroller, **I/O ports (Input/Output ports)** are used to connect the microcontroller with external devices like LEDs, switches, sensors, displays, etc. Each port consists of **8 pins**, and each pin can be individually configured as **input or output**.

Each I/O port in AVR has:

8 pins (e.g., PB0–PB7 for PORTB)

Three associated registers:

- **DDRx** (Data Direction Register)
- **PORTx** (Data Register)
- **PINx** (Input Pins Register)

Advantages of AVR I/O Ports

- Bit-level control (each pin independent)
- Internal pull-up resistors
- Fast operation (direct hardware access)
- Flexible configuration

Limitations

- Limited current capacity (~20mA per pin)
- Requires external drivers for heavy loads
- Limited number of ports

❖ Timers in AVR – Detailed Explanation

In the AVR Microcontroller, **timers** are hardware modules used to measure time, generate delays, count events, and create waveforms (like PWM). They are essential for real-time embedded applications.

1. What is a Timer?

A **timer** is a register that increments (or counts) based on a clock signal.

- If counting **internal clock pulses** → Timer
- If counting **external events** → Counter

👉 Hence often called **Timer/Counter**

Applications of AVR Timers

- 🕒 Time delay generation
- 📏 Event counting
- 🔔 Interrupt generation
- 🔊 PWM signal generation
- ⚙️ Motor control
- 📡 Frequency measurement

Advantages

- Accurate timing

- Hardware-based (no CPU load)
- Supports interrupts
- Multiple modes of operation

Limitations

- Limited number of timers
- Requires proper configuration
- Overflow handling needed

❖ USART in AVR – Detailed Explanation

In the AVR Microcontroller, **USART (Universal Synchronous/Asynchronous Receiver Transmitter)** is a hardware module used for **serial communication** between the microcontroller and external devices (like PCs, sensors, GSM modules, etc.).

What is USART?

USART is a communication interface that supports:

- **Asynchronous communication** (most common → UART mode)
 - **Synchronous communication** (with clock signal)
- 👉 It converts **parallel data** ↔ **serial data** for transmission over a single line.

Main Components:

- **Transmitter (TX)**
- **Receiver (RX)**
- **Baud Rate Generator**
- **Shift Registers**
- **Control & Status Registers**

Modes of Operation

◆ 1. Asynchronous Mode (UART)

- No clock signal
- Uses **baud rate** for timing
- Most widely used

◆ 2. Synchronous Mode

- Uses external clock
- Faster communication

USART Data Frame Format

A typical frame consists of:

Field	Description
Start Bit	Indicates beginning (LOW)
Data Bits	5–9 bits of data
Parity Bit	Error detection (optional)
Stop Bit(s)	End of frame (HIGH)

Important USART Registers

◆ UDR (USART Data Register)

- Holds transmitted/received data

◆ UCSRA (Control & Status Register A)

- Flags:
 - RXC → Receive complete
 - TXC → Transmit complete
 - UDRE → Data register empty

◆ UCSRB (Control & Status Register B)

- Enables:
 - Receiver (RXEN)
 - Transmitter (TXEN)
 - Interrupts

◆ UCSRC (Control & Status Register C)

- Sets:
 - Data size
 - Parity
 - Stop bits

◆ UBRR (Baud Rate Register)

- Sets communication speed

👉 Formula:

$$\text{Baud Rate} = \frac{F_{CPU}}{16 \times (UBRR + 1)}$$

Applications of USART

- Serial communication with PC
- Interfacing with modules (Bluetooth, GSM, GPS)
- Debugging (via serial monitor)
- Data logging systems

Advantages

- Simple communication
- Requires only 2 wires (TX, RX)
- Supports interrupt-based communication
- Flexible configuration

Limitations

- Slower than parallel communication
- Baud rate mismatch causes errors
- Limited range without additional hardware

❖ Interrupt Structure in AVR – Detailed Explanation

In the AVR Microcontroller, an **interrupt** is a mechanism that temporarily halts the normal execution of a program to **respond to an urgent event** (like a timer overflow, external signal, or data reception).

What is an Interrupt?

An interrupt:

- Stops the current program execution
- Transfers control to a special function called **ISR (Interrupt Service Routine)**
- After execution, returns to the main program

👉 Helps in **real-time processing** without constant polling

Types of Interrupts in AVR

◆ 1. External Interrupts

- Triggered by external signals on pins (INT0, INT1)
- Used for switches, sensors

◆ 2. Internal Interrupts

Generated by internal peripherals:

- Timer overflow
- ADC conversion complete
- USART transmit/receive
- SPI, I2C events

Advantages of Interrupts

- ⚡ Fast response to events
- 🔄 Efficient CPU utilization
- 🕒 Better real-time performance
- ❌ Eliminates continuous polling

Limitations

- Complex programming
- Risk of interrupt conflicts
- Requires careful handling of shared resources

Applications

- Real-time systems
- Embedded control systems
- Communication handling (USART, SPI)
- Event-driven systems